Gaming Castle

Online Gaming Centre Management System

System Design Specification (SDS)

By

Kirisigan – SE/2021/007

Nirojan – SE/2021/014

Thivya – SE/2021/022

Jashvanth – SE/2021/055

A report submitted in partial fulfilment of the requirements for the degree of

Bachelor of Science Honours in Software Engineering (B.Sc.SE)

Software Engineering Teaching Unit

Faculty of Science

University of Kelaniya

Sri Lanka

2026

Version History

Version No.	Date	Author(s)	Summary of Changes
01	2026/04/10	Kirisigan, Nirojan, Thivya, Jashvanth	Initial draft of SDS document
02	2026/04/15	Kirisigan,Jashvanth, Thivya,Nirojan	All diagrams completed, security and deployment sections finalised

Table 1: Version History

Abstract

This System Design Specification (SDS) defines the complete technical design for the Gaming Castle Online Gaming Centre Management System — a web-based platform developed to manage PS5 slot bookings, tournament registrations, payment processing, and loyalty rewards for a gaming hub located in Nelliyadi, Sri Lanka.

The system is designed using a Microservices-based architecture comprising five independently deployable services, each internally structured following the Model-View-Controller (MVC) pattern, and deployed on Amazon Web Services (AWS) via Docker containerisation. A PostgreSQL relational database is adopted under a database-per-service strategy, with the schema conforming to the Third Normal Form (3NF).

Security is enforced through JWT-based authentication, Role-Based Access Control (RBAC), bcrypt password hashing, TLS 1.2/1.3 encryption in transit, and AES-256 encryption at rest. The frontend is built using Angular, conforming to WCAG 2.1 Level AA accessibility standards across all device types. Deployment is automated through a GitHub Actions CI/CD pipeline across Development, Staging, and Production environments.

This document is intended for the development team, academic evaluators, and the client, and is to be read in conjunction with the Software Requirements Specification (SRS) for the Gaming Castle system.

Table of Contents

List of Tables

List of Figures

Chapter 1 Introduction

1.1 Purpose

This System Design Specification (SDS) document defines the technical design decisions for the Gaming Castle Online Gaming Centre Management System. This document serves as the primary technical reference for the development team and describes the system architecture, database design, class structure, sequence diagrams, UI/UX design, security design, and deployment strategy.

The intended audience of this document includes the development team, academic evaluators, and the client. This document is to be read in conjunction with the Software Requirements Specification (SRS) for Gaming Castle.

1.2 Scope

The Gaming Castle system is a web-based platform designed to manage slot bookings, tournament registrations, payment processing, loyalty rewards, and administrative operations for a PS5 gaming hub in Nelliyadi, Sri Lanka. This SDS document provides the complete technical design required to implement the system as defined in the SRS.

1.3 Definitions, Acronyms and Abbreviations

Term / Acronym	Definition
SDS	System Design Specification
SRS	Software Requirements Specification

MVC	Model-View-Controller architectural pattern
API	Application Programming Interface
ER Diagram	Entity-Relationship Diagram
JWT	JSON Web Token
RBAC	Role-Based Access Control
CI/CD	Continuous Integration / Continuous Deployment
WCAG	Web Content Accessibility Guidelines
TLS	Transport Layer Security
DXA	Document Exchange Architecture (unit of measurement in Word)
VPC	Virtual Private Cloud
CDN	Content Delivery Network
SPA	Single Page Application

Table 2: Definitions, Acronyms and Abbreviations

1.4 Document Overview

This document is organised as follows:

- Chapter 1 – Introduction: Provides the purpose, scope, definitions, and overview.
- Chapter 2 – Architectural Design: Describes the high-level architecture and justification.
- Chapter 3 – Database Design: Presents the ER diagram and normalisation.
- Chapter 4 – Class Diagram: Details classes, attributes, methods, and relationships.
- Chapter 5 – Sequence Diagrams: Illustrates interaction flows for all major use cases.
- Chapter 6 – UI/UX Design: Covers wireframes, user flow, and accessibility.
- Chapter 7 – Security Design: Defines authentication, authorisation, and data protection.
- Chapter 8 – Deployment Design: Specifies the hosting plan, CI/CD pipeline, and environments.
- Chapter 9 – References: Lists all cited sources and standards.

Chapter 2 Architectural Design

2.1 High-Level Architecture

The Gaming Castle system shall be implemented using a Microservices-based Architecture with a layered internal structure within each service. The system comprises five independently deployable microservices, each responsible for a distinct business domain.

The five core microservices are:

- User Service – Manages customer and administrator account registration, authentication, and profile management.
- Booking Service – Handles PS5 slot availability, online bookings, walk-in bookings, and cancellation or rescheduling.
- Payment Service – Processes online payments via a payment gateway and records cash payments made by the administrator.
- Tournament Service – Manages tournament creation, participant registration, bracket generation, and status updates.
- Loyalty Service – Tracks loyalty point accrual, redemption, and balance queries for registered customers.

Each microservice internally follows the Model-View-Controller (MVC) layered pattern with a defined separation of concerns across the Controller layer (REST API endpoints), Service layer (business logic), Repository layer (database access), and Model layer (data entities).

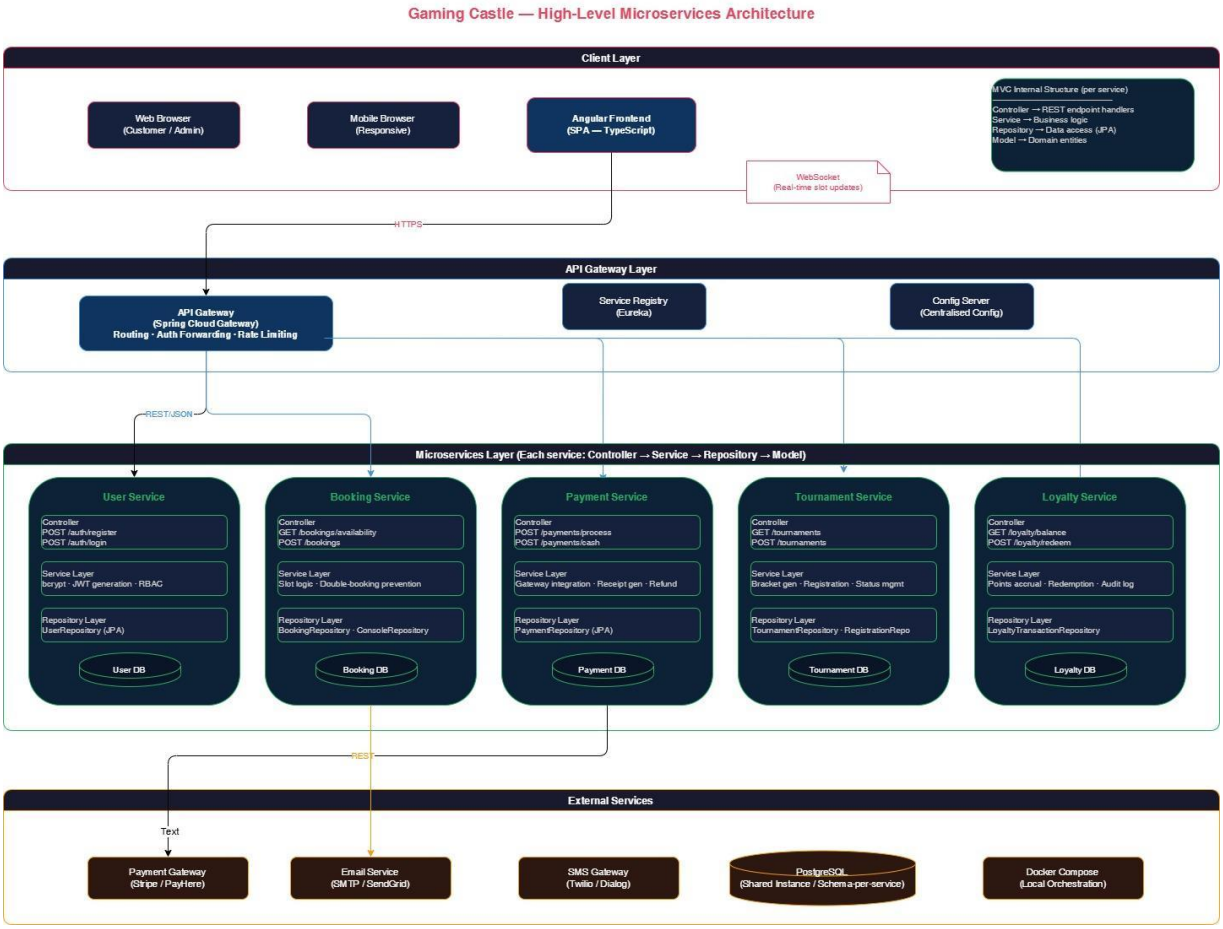

Figure 1: High-Level Architecture Diagram

2.2 Cloud-Native Architecture

The system is designed as a cloud-native application to ensure scalability, resilience, and ease of deployment. The following cloud-native design principles are applied:

- Containerisation: Each microservice shall be packaged as a Docker container, ensuring environment consistency across development, staging, and production.
- API Gateway: An API Gateway (e.g., Spring Cloud Gateway) shall serve as the single entry point for all client requests, handling routing, authentication forwarding, and rate limiting.
- Service Registry: A service registry (e.g., Eureka) shall be used for microservice discovery, enabling dynamic routing and fault tolerance.

- Centralised Configuration: A configuration server shall manage environment-specific configuration across all microservices.
- Horizontal Scaling: Each service may be scaled independently by adding container instances behind a load balancer, supporting the NFR requirement of a 3x user load increase without redesign.

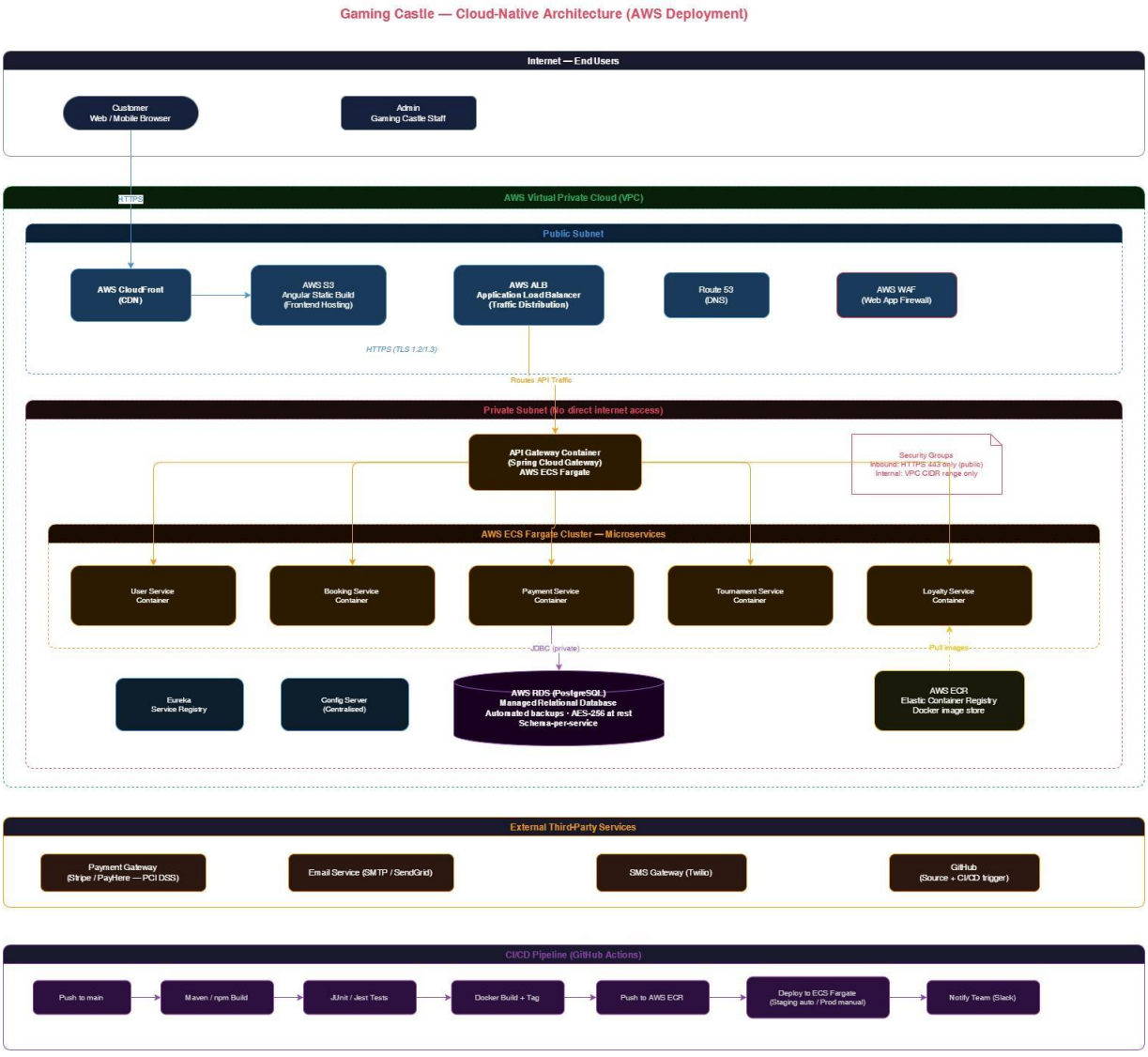

Figure 2: Cloud-Native Architecture

2.3 Justification for Architecture Choice

The Microservices architecture was selected over monolithic and layered alternatives for the following reasons:

Criterion	Microservices (Selected)	Monolithic (Rejected)
Independent Deployment	Each service deployed independently	Full redeployment required
Scalability	Per-service horizontal scaling	Scale entire application
Fault Isolation	Failure in one service does not cascade	Single point of failure
Team Alignment	Each member owns a domain service	Shared codebase conflicts
Maintainability	Small, focused, testable services	Large codebase increases complexity

Table 3: Architecture Comparison

2.4 Scalability and Maintainability Considerations

- Each microservice may be independently upgraded, replaced, or scaled without affecting others.
- Docker Compose shall be used for local development orchestration; Kubernetes or AWS ECS may be introduced for production deployment.

- A shared database-per-service pattern shall be adopted, ensuring loose coupling between data stores.
- All inter-service communication shall be conducted via RESTful HTTP APIs using JSON payloads.
- WebSocket connections shall be supported for real-time slot availability updates from the Booking Service to the frontend.

2.5 Technology Stack

- Frontend: Angular
- Backend: Spring Boot (Microservices)
- Database: PostgreSQL
- Authentication: JWT
- Containerization: Docker
- CI/CD: GitHub Actions
- Cloud: AWS (ECS, RDS, S3)

Chapter 3 Database Design

3.1 Overview

A relational database management system (PostgreSQL) shall be used as the primary data store for the Gaming Castle system. A database-per-service strategy is adopted for the microservices architecture. Each service maintains its own schema within a shared PostgreSQL instance, allowing independent data management while avoiding the operational complexity of fully separate database servers in the initial deployment.

3.2 Entity-Relationship Diagram

The following key entities and relationships constitute the data model for the Gaming Castle system. The ER diagram below illustrates the full relational structure, cardinalities, and foreign key relationships.

Entity	Primary Key	Foreign Keys / Relationships
User	user_id	None (root entity)
Console	console_id	None (managed by admin)
Booking	booking_id	user_id → User, console_id → Console
Payment	payment_id	booking_id → Booking
Tournament	tournament_id	created_by → User (Admin)
TournamentRegistration	registration_id	tournament_id → Tournament, user_id → User, payment_id → Payment
LoyaltyTransaction	transaction_id	user_id → User, booking_id → Booking
Review	review_id	user_id → User, booking_id → Booking
Game	game_id	Console_id→ Console

GameConsole	game_id,console_id	game_id→Game, console_id→Console

Table 4: Entity-Relationship Summary

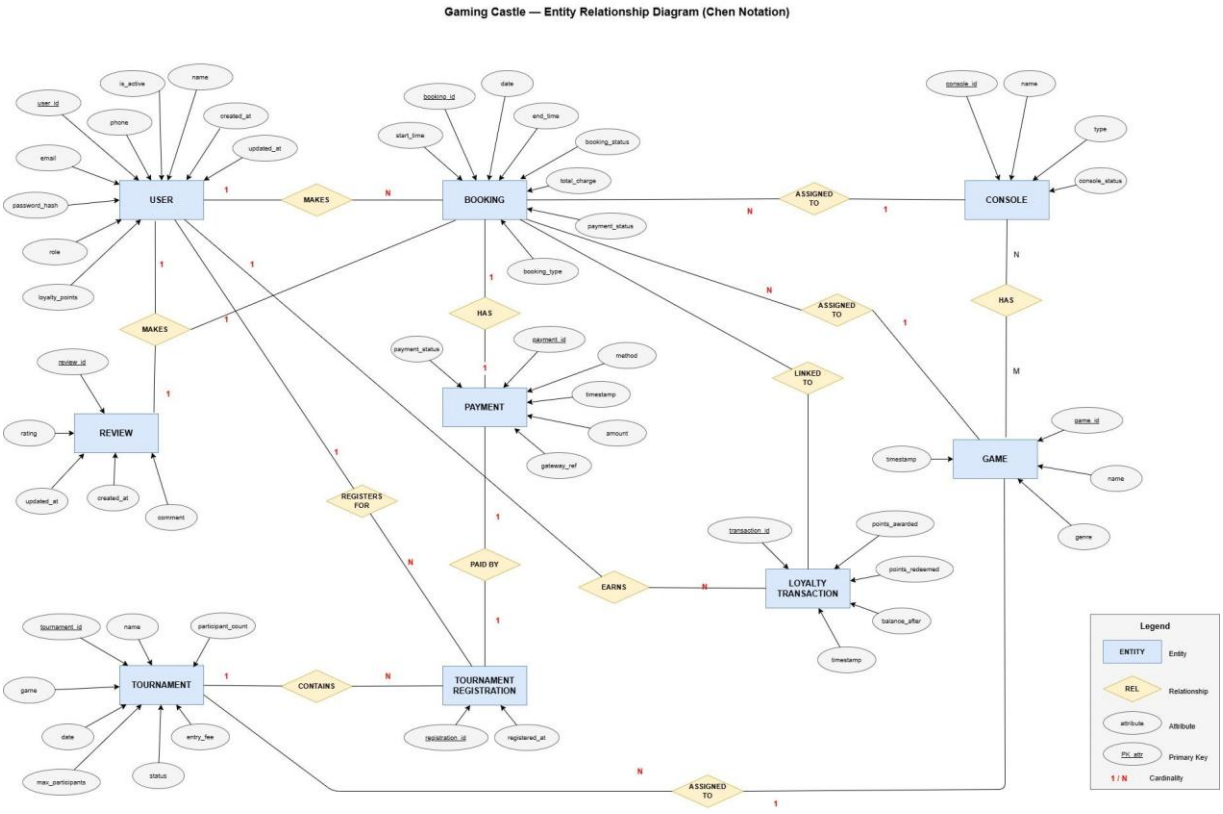

Figure 3: Entity-Relationship Diagram

3.3 Key Entity Attributes

Entity	Attributes	Notes
User	user_id, name, email, phone, password_hash, role (CUSTOMER/ADMIN), loyalty_points, is_active, created_at, updated_at	bcrypt hashed password; role enum
Console	console_id, name, type (PS5), console_status (AVAILABLE/OCCUPIED/MAINTENANCE)	Status updated on booking
Booking	booking_id, user_id, console_id, date, game_id, start_time, end_time, total_charge, payment_status, booking_type (ONLINE/WALKIN), booking_status (CONFIRMED/CANCELLED)	Double-booking prevented by unique constraint
Payment	payment_id, booking_id, amount, method (ONLINE/CASH), payment_status, gateway_ref, timestamp	gateway_ref null for cash
Tournament	tournament_id,game_id, name, game, date, entry_fee, max_participants, participant_count, status (UPCOMING/ONGOING/COMPLETED/CANCELLED), created_by	Auto-closes registration when full
Tournament Registration	registration_id, tournament_id, user_id, payment_id, registered_at	Unique(tournament_id, user_id)
Loyalty Transaction	transaction_id, user_id, points_awarded, points_redeemed, balance_after, booking_id, timestamp	Full audit trail of points history

Review	review_id,booking_id, user_id,rating,comment,created_at,updated_at	After complete game user can rating and review
Game	game_id,name,genre,created_at, updated_at	While booking show game available on that console.
Console Game	game_id,console_id,created_at,updated_at	Create a game by admin in particular console.

Table 5: Key Entity Attributes

3.4 Normalisation

The database schema is designed to conform to the Third Normal Form (3NF) to eliminate data redundancy and ensure referential integrity. The following principles are applied:

- First Normal Form (1NF): All attributes contain atomic values. No repeating groups are used. Each table has a primary key.
- Second Normal Form (2NF): All non-key attributes are fully functionally dependent on the entire primary key. No partial dependencies exist in any table.
- Third Normal Form (3NF): All non-key attributes depend directly on the primary key and not on other non-key attributes. Transitive dependencies are eliminated. For example, customer name and email are stored only in the User table and referenced via foreign keys in Booking and TournamentRegistration.
- Indexes: Indexes shall be applied on frequently queried foreign keys (user_id, console_id, booking_id) and on date/time columns in the Booking table to support performant calendar queries.

Chapter 4 Class Diagram

4.1 Overview

The class diagram below defines the main domain model classes for the Gaming Castle system. The classes correspond to the data entities and service-layer components identified in the architectural and database design. Each class includes its attributes (with visibility and type), methods, and relationships.

4.2 Core Domain Classes

Class	Key Attributes	Key Methods
User	- userId: UUID - name: String - email: String - phone: String - passwordHash: String - role: Role - loyaltyPoints: int - isActive: boolean	+ register(): void + login(email, password): Token + resetPassword(token, newPwd): void + updateProfile(details): void + deactivate(): void
Console	- consoleId: UUID - name: String - type: String - status: ConsoleStatus - games: List<Game>	+ getAvailableSlots(date): List<TimeSlot> + markOccupied(slot): void + markAvailable(slot): void

Booking	- bookingId: UUID - userId: UUID - gameId: UUID - consoleId: UUID - date: LocalDate - startTime: LocalTime - endTime: LocalTime - totalCharge: BigDecimal - paymentStatus: PaymentStatus - bookingType: BookingType - bookingStatus: BookingStatus	+ create(details): Booking + cancel(): void + reschedule(newSlot): void + calculateCharge(): BigDecimal + confirmPayment(): void
Payment	- paymentId: UUID - bookingId: UUID - amount: BigDecimal - method: PaymentMethod - status: PaymentStatus - gatewayRef: String - timestamp: DateTime	+ processOnlinePayment(): void + recordCashPayment(): void + generateReceipt(): Receipt + refund(): void
Tournament	- tournamentId: UUID - name: String - gameId: UUID - date: LocalDate - entryFee: BigDecimal - maxParticipants: int - participantCount: int - status: TournamentStatus - createdBy: UUID	+ create(details): Tournament + publish(): void + cancel(): void + updateStatus(status): void + generateBracket(): Bracket + isRegistrationOpen(): boolean
TournamentRegistration	- registrationId: UUID - tournamentId: UUID - userId: UUID - paymentId: UUID - registeredAt: DateTime	+ register(user, tournament): void + cancelRegistration(): void + notifyParticipant(): void
LoyaltyTransaction	- transactionId: UUID - userId: UUID - pointsAwarded: int - pointsRedeemed: int - balanceAfter: int - bookingId: UUID - timestamp: DateTime	+ awardPoints(booking): void + redeemPoints(amount): void + getBalance(userId): int + getHistory(userId): List<LoyaltyTransaction>

Review	- reviewId: UUID - bookingId: UUID - userId: UUID - rating: int - comment: String - createdAt: DateTime - updatedAt: DateTime	+ addReview(bookingId, userId, rating, comment): Review + updateReview(reviewId, rating, comment): void + deleteReview(reviewId): void + getReviewByBooking(bookingId): Review + getUserReviews(userId): List<Review>
Game	- gameId: UUID - name: String - genre: Genre - createdAt: DateTime - updatedAt: DateTime	+ create(): void, + update(): void, + delete(): void, + getByConsole(consoleId): List<Game>

Table 6: Core Domain Classes

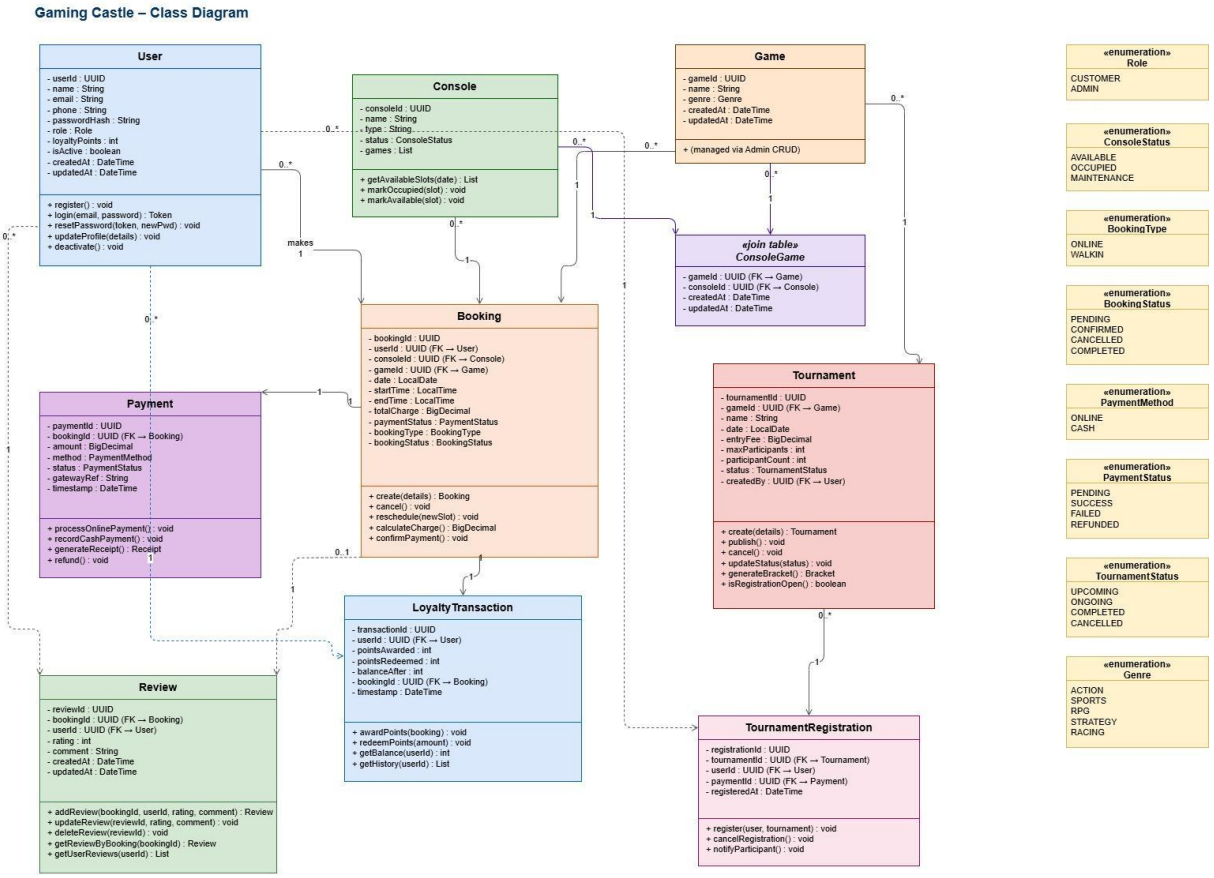

Figure 4: Class Diagram

4.3 Relationships

- User – Booking: One-to-Many. A single user may hold multiple bookings.
- Console – Booking: One-to-Many. A single console may be associated with multiple bookings across different time slots.
- Console – Game: Many-to-Many. A console can support multiple games, and a game can be available on multiple consoles.
- Game – Booking: One-to-Many. A single game may be associated with multiple bookings, while each booking is for one selected game.
- Booking – Review: One-to-One. Each booking can have at most one review.
- User – Review: One-to-Many. A user may write multiple reviews for different bookings.
- Booking – Payment: One-to-One. Each booking is associated with exactly one payment record.
- User – TournamentRegistration: One-to-Many. A user may register for multiple tournaments.
- Tournament – TournamentRegistration: One-to-Many. A tournament may have multiple registered participants.
- Game – Tournament: One-to-Many. A single game can have multiple tournaments, while each tournament is associated with one game.
- Booking – LoyaltyTransaction: One-to-One. Each completed booking generates one loyalty transaction record.

4.4 Enumerations

Enumeration	Values
Role	CUSTOMER, ADMIN
ConsoleStatus	AVAILABLE, OCCUPIED, MAINTENANCE
BookingType	ONLINE, WALKIN
BookingStatus	PENDING, CONFIRMED, CANCELLED, COMPLETED
PaymentMethod	ONLINE, CASH
PaymentStatus	PENDING,SUCCESS, FAILED, REFUNDED
TournamentStatus	UPCOMING, ONGOING, COMPLETED, CANCELLED
Genre	ACTION,SPORTS,RPG,STRATEGY,RACING

Table 7: Domain Enumerations

Chapter 5 Sequence Diagrams

5.1 Overview

This chapter presents sequence diagrams for all major use cases of the Gaming Castle system. Each diagram describes the temporal interaction between actors and system components, including the frontend (Angular), API Gateway, relevant microservice, and the database.

5.2 UC-01: Register Account

The registration sequence involves client-side input submission, server-side field validation, uniqueness checking, password hashing, account creation, and confirmation email dispatch.

Step	From	To	Message / Action
1	Customer (Browser)	Angular Frontend	Submit registration form (name, email, phone, password)
2	Angular Frontend	API Gateway	POST /api/auth/register
3	API Gateway	User Service	Forward registration request
4	User Service	User DB	SELECT: check email uniqueness
5	User Service	User Service	Hash password using bcrypt
6	User Service	User DB	INSERT: new user record, role=CUSTOMER, loyalty_points=0
7	User Service	Email Service	Send confirmation email
8	User Service	Angular Frontend	201 Created – Redirect to Login page

Table 8: Sequence – UC-01 Register Account

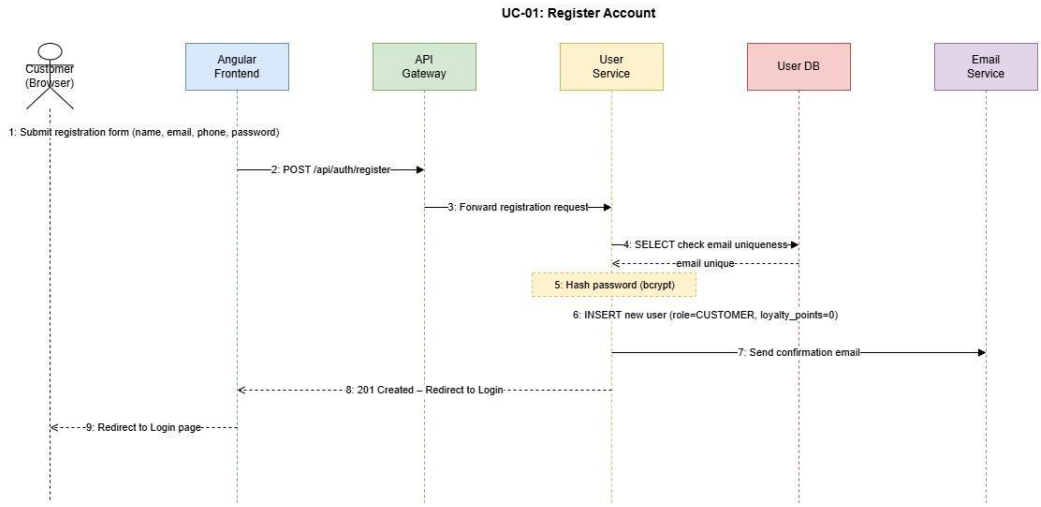

Figure 5: Sequence – UC-01 Register Account

5.3 UC-02: Login to System

Step	From	To	Message / Action
1	User (Browser)	Angular Frontend	Submit login form (email, password)
2	Angular Frontend	API Gateway	POST /api/auth/login
3	API Gateway	User Service	Forward login request
4	User Service	User DB	SELECT: fetch user by email; validate bcrypt hash

5	User Service	User Service	Check failed_attempts; lock account if >= 5
6	User Service	User Service	Determine role (CUSTOMER or ADMIN); generate JWT
7	User Service	Angular Frontend	200 OK – return JWT token + role
8	Angular Frontend	Browser	Redirect to Customer Dashboard or Admin Dashboard based on role

Table 9: Sequence – UC-02 Login to System

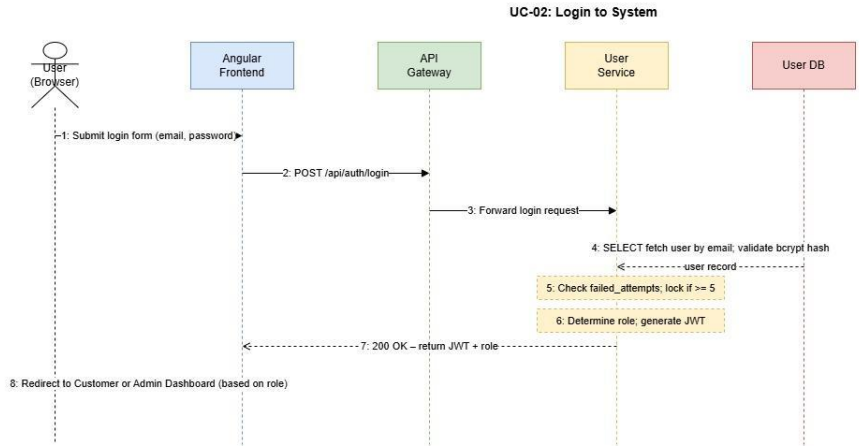

Figure 6: Sequence – UC-02 Login to System

5.4 UC-03: Book Gaming Slot

Step	From	To	Message / Action

1	Customer	Angular Frontend	Navigate to Book a Slot; select date, time, console, game
2	Angular Frontend	Booking Service	GET /api/bookings/availability?date=X&time=Y
3	Booking Service	Booking DB	SELECT: check no existing confirmed booking for console/slot
4	Booking Service	Angular Frontend	Return availability + booking summary with charge
5	Customer	Angular Frontend	Confirm booking; proceed to payment
6	Angular Frontend	Payment Service	POST /api/payments/process (bookingDetails, paymentMethod)
7	Payment Service	Payment Gateway	Send payment request; await gateway response
8	Payment Service	Booking Service	Payment confirmed: update booking status to CONFIRMED
9	Booking Service	Loyalty Service	Award loyalty points for completed booking

10	Booking Service	Notification Service	Send booking confirmation via email/SMS

Table 10: Sequence – UC-03 Book Gaming Slot

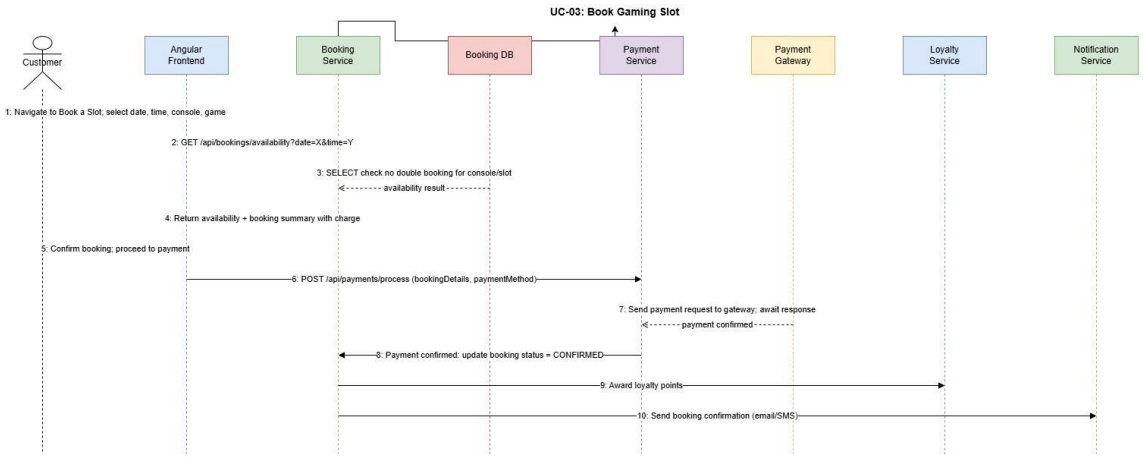

Figure 7: Sequence – UC-03 Book Gaming Slot

5.5 UC-05: Admin Book Slot for Walk-in Customer

Step	From	To	Message / Action
1	Admin	Angular Frontend	Navigate to Admin Booking panel; select slot, console, game
2	Angular Frontend	Booking Service	GET /api/admin/bookings/availability
3	Admin	Angular Frontend	Enter walk-in customer name; submit booking

4	Angular Frontend	Booking Service	POST /api/admin/bookings (booking details, type=WALKIN)
5	Booking Service	Booking DB	INSERT: booking record; mark slot as OCCUPIED
6	Admin	Angular Frontend	Record cash payment: enter amount received
7	Angular Frontend	Payment Service	POST /api/payments/cash (bookingId, amount)
8	Payment Service	Payment DB	INSERT: payment record (method=CASH); update revenue records
9	Payment Service	Angular Frontend	Generate and display receipt

Table 11: Sequence – UC-05 Admin Walk-in Booking

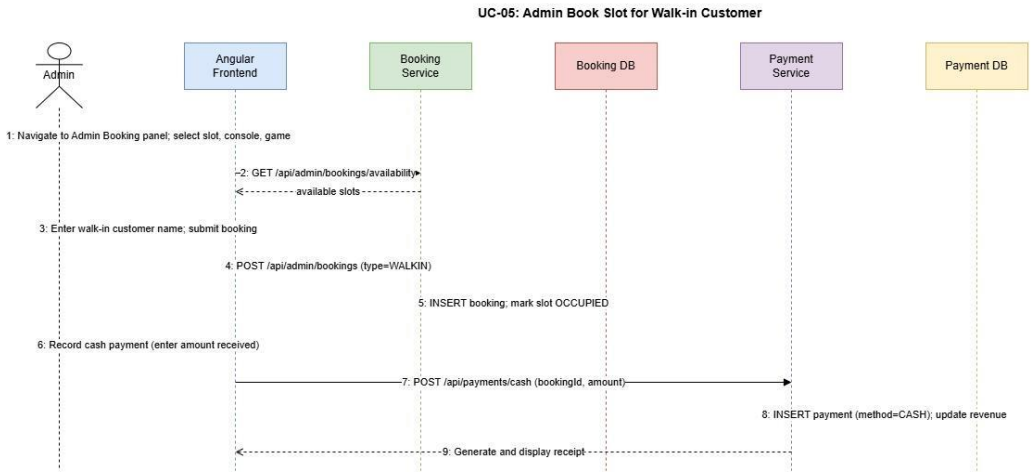

Figure 8: Sequence – UC-05 Admin Walk-in Booking

5.6 UC-08: Register for Tournament

Step	From	To	Message / Action
1	Customer	Angular Frontend	Navigate to Tournaments page; view available tournaments
2	Angular Frontend	Tournament Service	GET /api/tournaments?status=UPCOMING
3	Customer	Angular Frontend	Select tournament; click Register
4	Tournament Service	Tournament DB	Check: registration open? participant_count < max_participants?
5	Angular Frontend	Payment Service	POST /api/payments/process (entry fee)
6	Payment Service	Tournament Service	Payment confirmed: register participant; increment participant_count
7	Tournament Service	Notification Service	Send confirmation notification to customer

Table 12: Sequence – UC-08 Register for Tournament

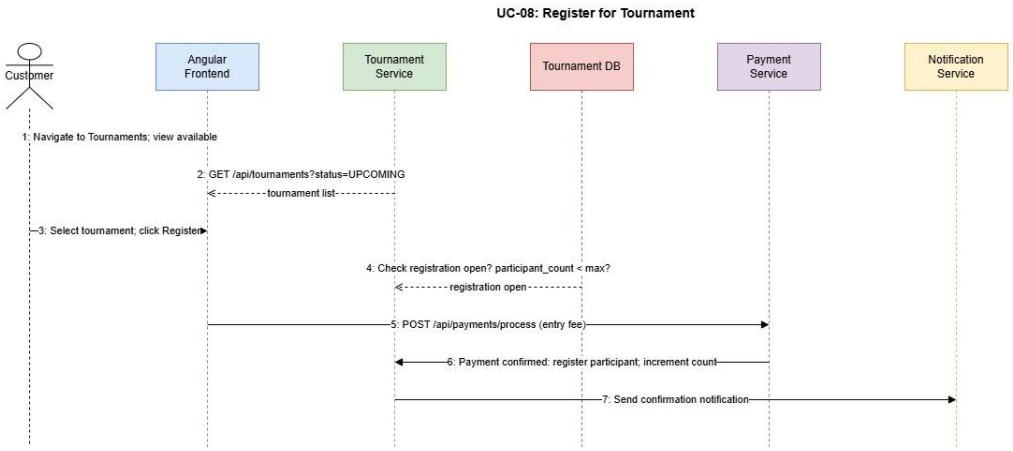

Figure 9: Sequence – UC-08 Register for Tournament

5.7 UC-11: Redeem Loyalty Points

Step	From	To	Message / Action
1	Customer	Angular Frontend	Proceed to payment for a booking
2	Angular Frontend	Loyalty Service	GET /api/loyalty/balance (userId)
3	Angular Frontend	Customer	Display available points and equivalent discount

4	Customer	Angular Frontend	Select 'Redeem Points'; view discounted total
5	Angular Frontend	Payment Service	POST /api/payments/process (amount after discount, loyaltyPointsUsed)
6	Payment Service	Loyalty Service	Deduct redeemed points from balance; log LoyaltyTransaction
7	Payment Service	Angular Frontend	Booking confirmed; updated loyalty balance displayed

Table 13: Sequence – UC-11 Redeem Loyalty Points

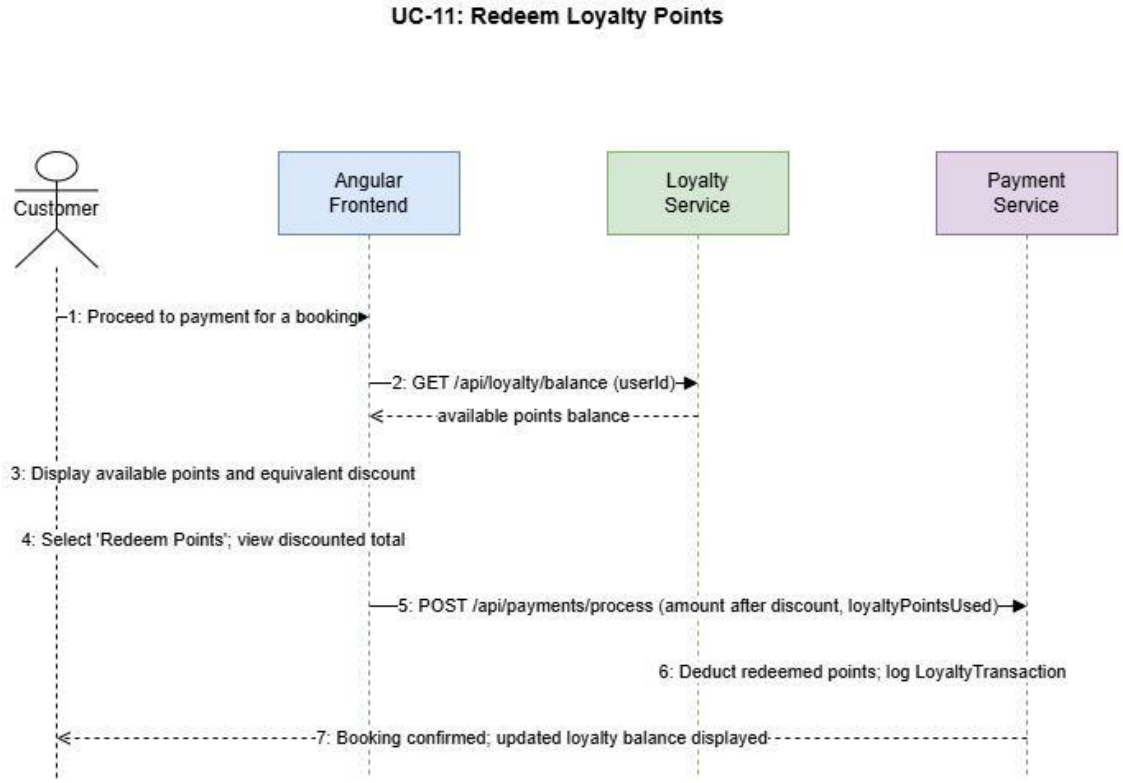

Figure 10: Sequence – UC-11 Redeem Loyalty Points

Chapter 6 UI/UX Design

6.1 Overview

The Gaming Castle web application shall provide a modern, responsive, and accessible user interface. The design follows a dark-themed gaming aesthetic that is consistent with the Gaming Castle brand identity. All pages shall render correctly on desktop, tablet, and mobile devices (minimum viewport width of 320px), in conformance with NFR07.

6.2 Design Principles

- Consistency: A unified colour scheme, typography (sans-serif headings, readable body text), and iconography shall be applied across all pages.
- Simplicity: Core customer workflows (slot booking, tournament registration, loyalty redemption) shall be completable in a maximum of 5 minutes by a first-time user.
- Feedback: All form submissions, payment processing stages, and booking confirmations shall provide visible status feedback via toast notifications or inline messages.
- Accessibility: The interface shall comply with WCAG 2.1 Level AA, ensuring usability for users with visual or motor impairments.

6.3 Key Screens and Wireframe Descriptions

6.3.1 Customer – Home Page

The home page shall display a hero section with a call-to-action 'Book a Slot' button, a featured upcoming tournament card, and a loyalty points balance widget for authenticated users. The top navigation bar shall include links to Book a Slot, Tournaments, My Bookings, Loyalty Rewards, and Profile.

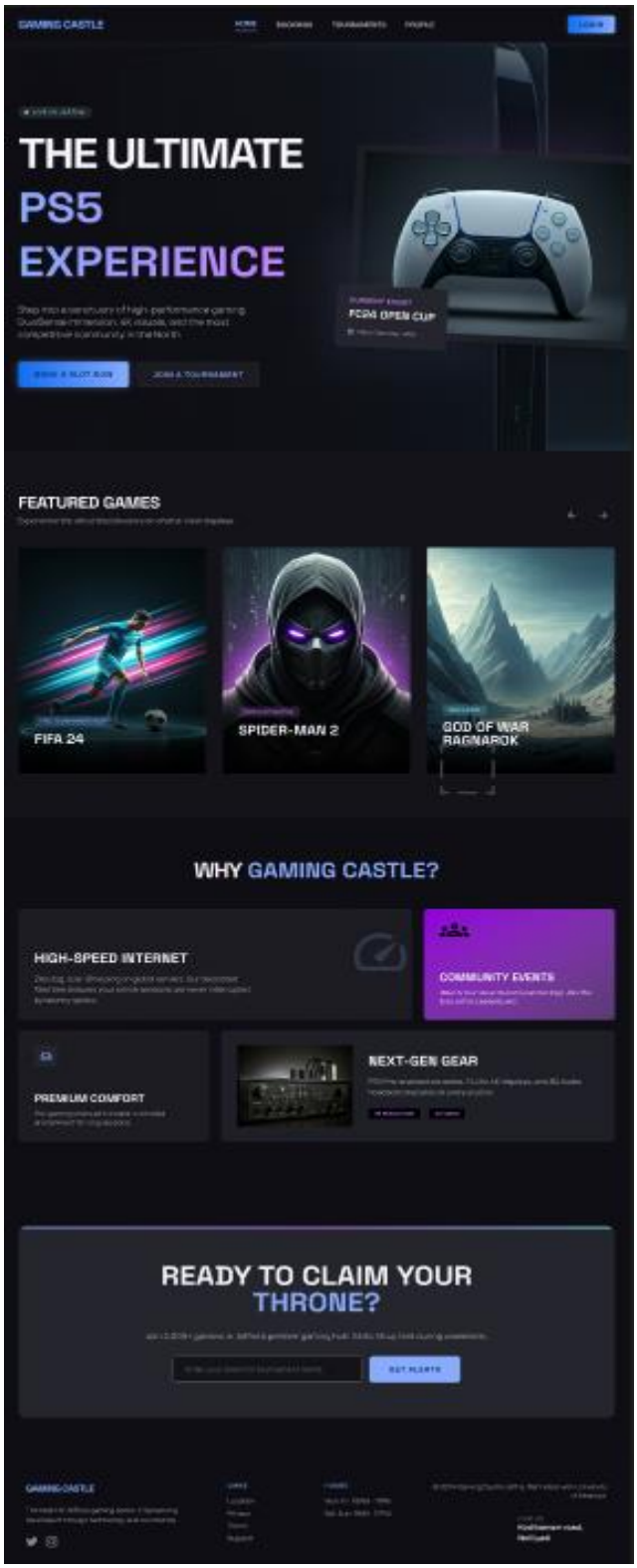

Figure 11: UI/UX Design - Customer – Home Page

6.3.2 Customer – Slot Booking Page

The slot booking page shall present an interactive calendar date picker, time slot grid showing available and occupied slots with colour coding (green = available, red = occupied), a console and game selector, a real-time booking summary panel, and a payment and confirm button. Unavailable slots shall be visually disabled and not selectable.

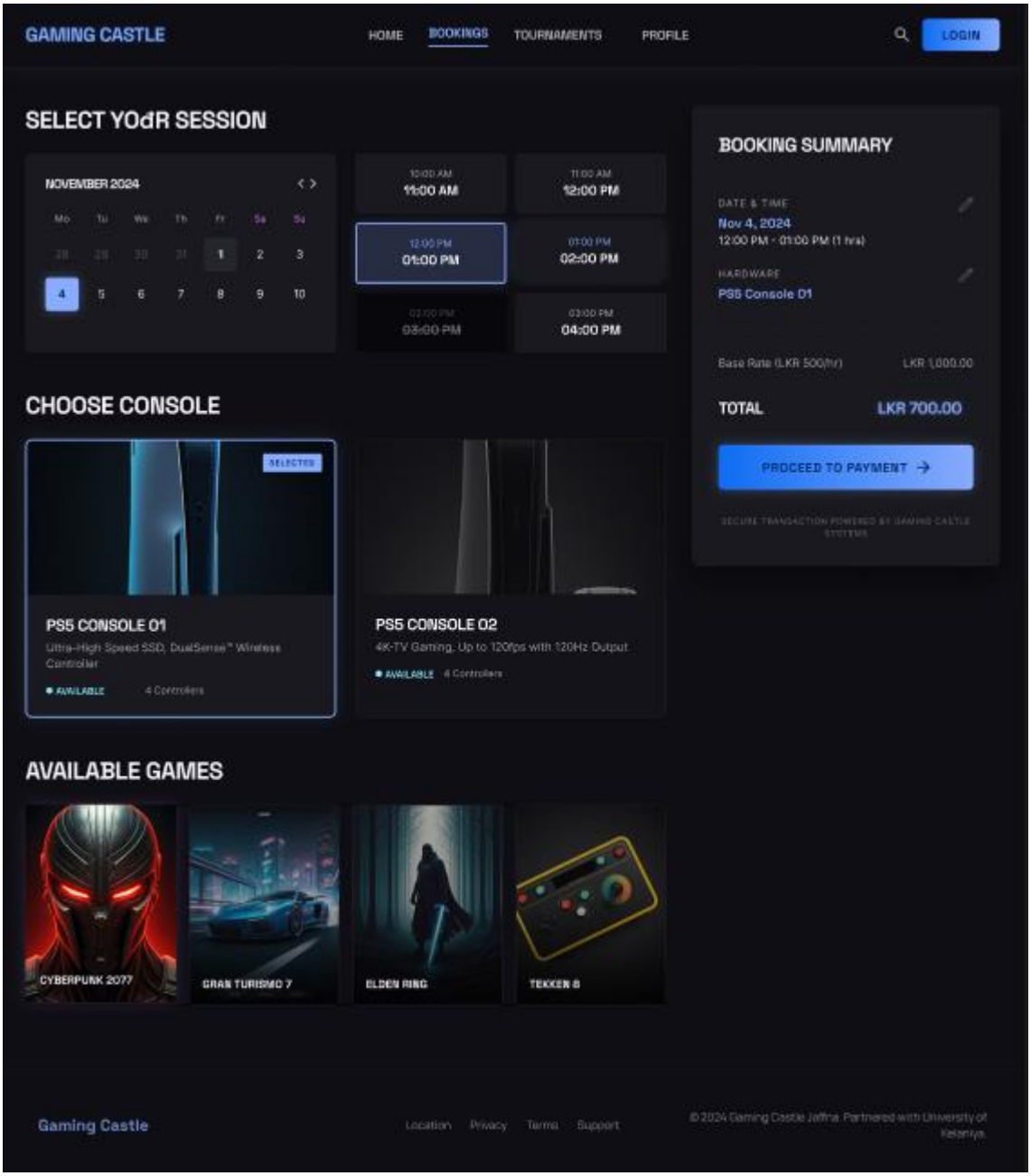

Figure 12: UI/UX Design - Customer – Slot Booking Page

6.3.3 Customer – Payment Page

The payment page shall display the booking summary with the applicable charge, a loyalty points redemption toggle showing the available points balance and resulting discount, a payment method selector (card or gateway), a secure card entry form, and an order total with a confirm payment button.

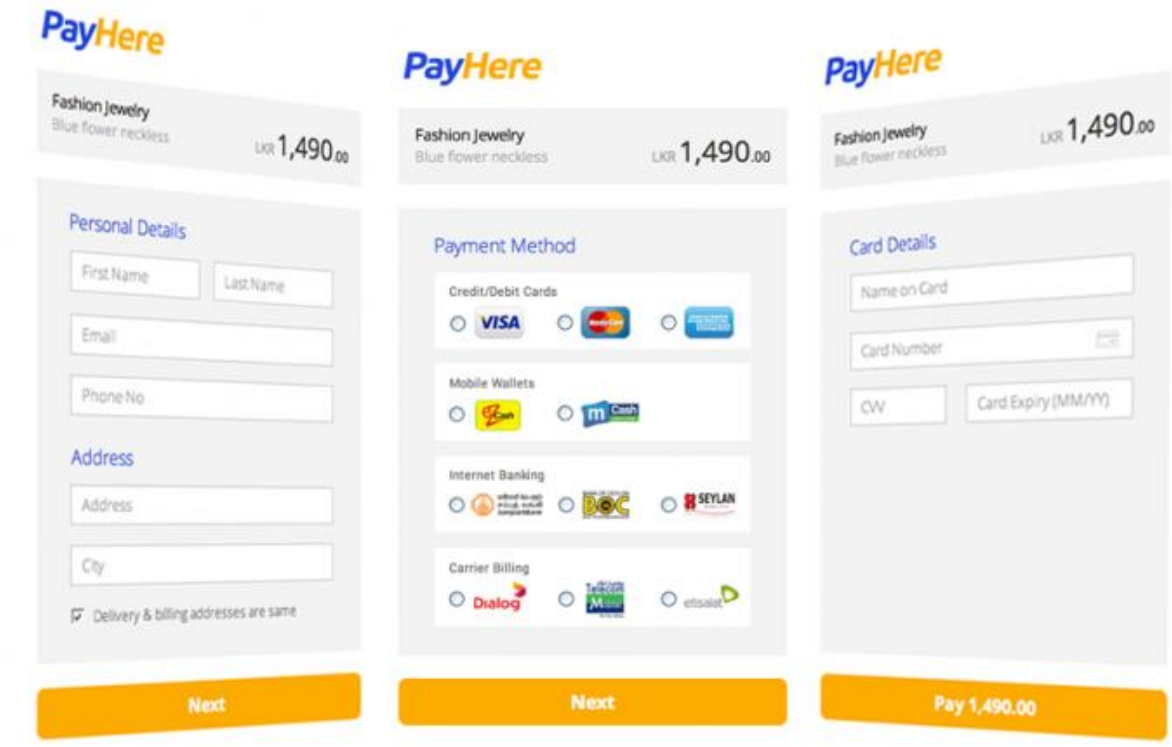

Figure 13: UI/UX Design - Customer – Payment Page

6.3.4 Customer – Tournament Page

The tournaments page shall list all available and upcoming tournaments in card format, showing the game title, date, entry fee, remaining spots, and a register button. Closed or full tournaments shall be displayed with a disabled state and a status badge.

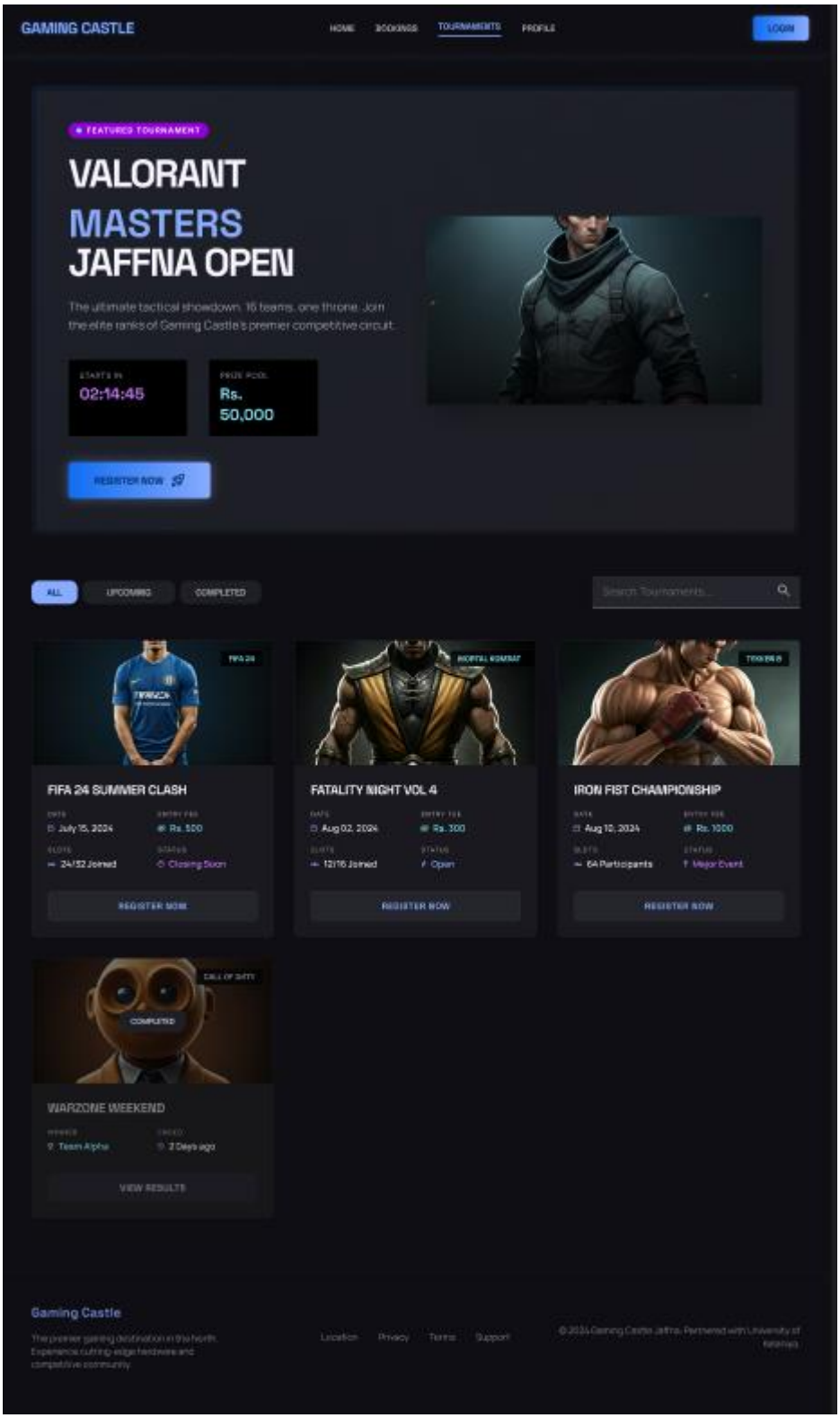

Figure 14: UI/UX Design - Customer – Tournament Page

6.3.5 Admin – Dashboard

The admin dashboard shall provide a summary panel with four key widgets: Today's Bookings (count and revenue), Active Sessions (live console occupancy), Upcoming Tournaments, and Total Registered Users. A left-side navigation panel shall provide access to Bookings, Tournaments, Users, Payments, and Reports sections.

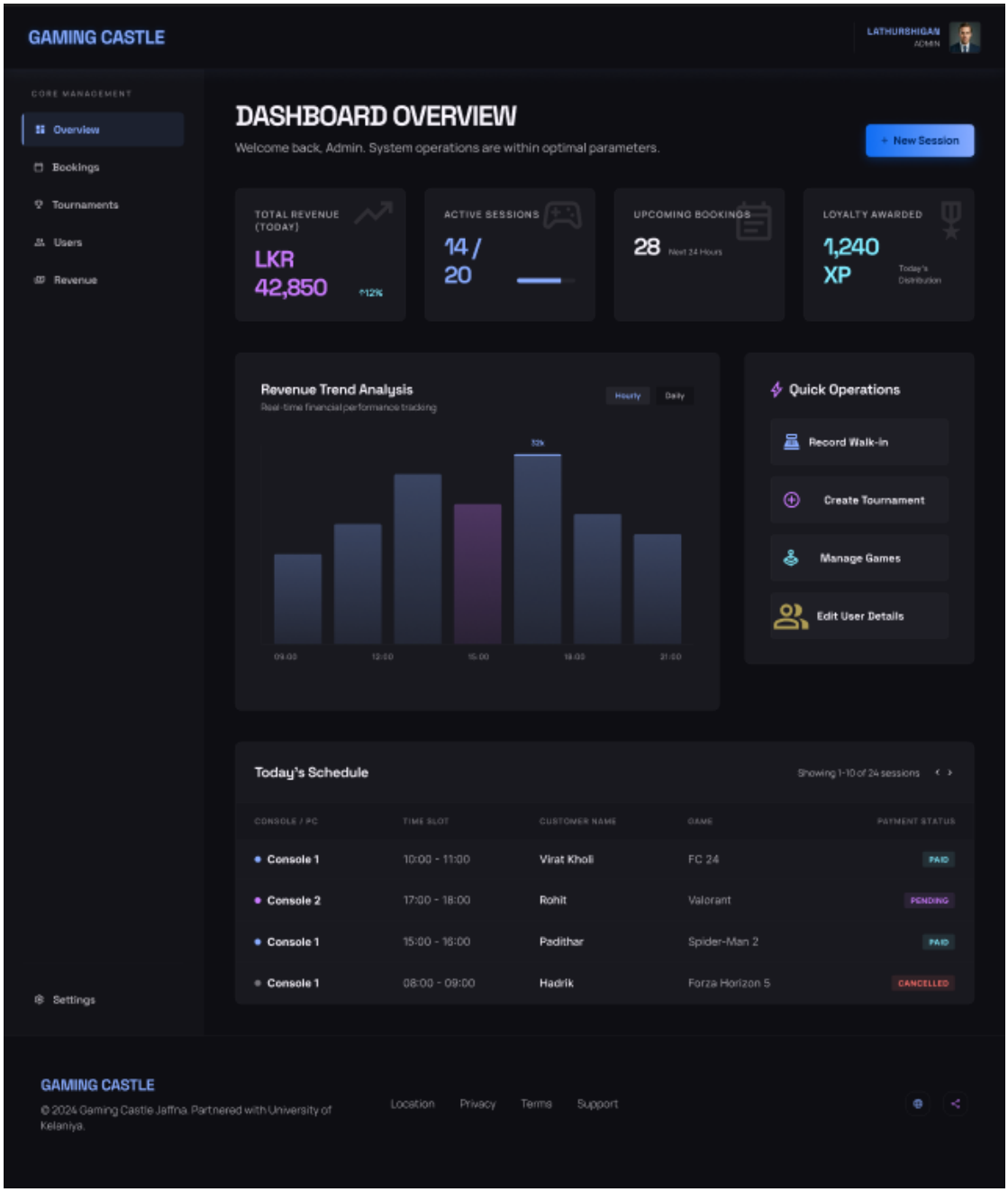

Figure 15: UI/UX Design - Admin – Dashboard

6.3.6 Admin – Booking Management

The admin booking management page shall display a searchable and filterable table of all bookings (date, customer, console, payment status), an inline walk-in booking form, and action buttons for cancel and reschedule. A booking detail panel shall be accessible on row selection.

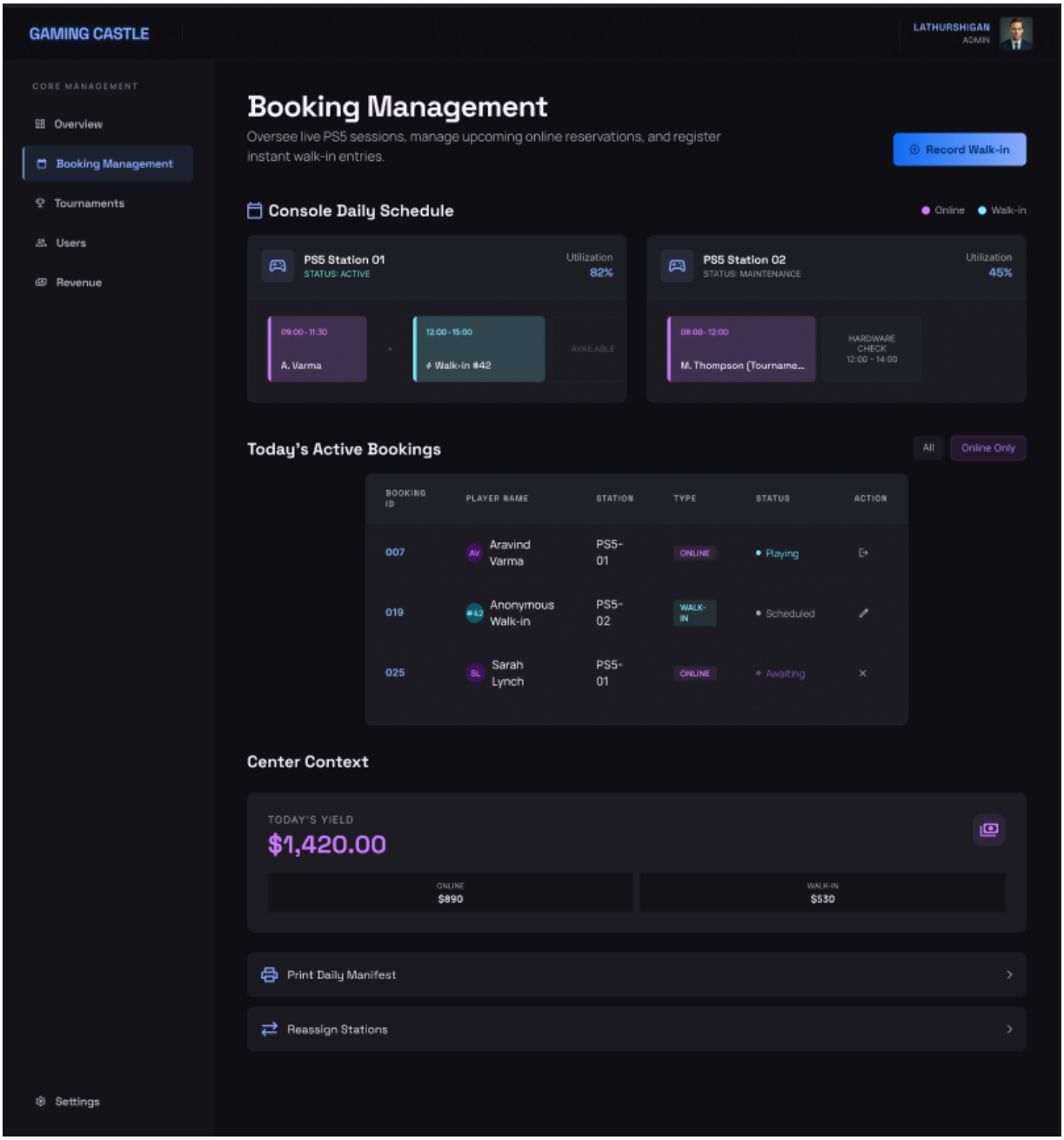

Figure 16: UI/UX Design - Admin – Booking Management

6.4 Navigation Flow

User Role	Navigation Path
Customer	Login → Home → Book Slot → Payment → Confirmation
Customer	Login → Tournaments → Tournament Detail → Register → Payment
Customer	Login → Profile → Loyalty Rewards → Redeem (on Booking page)
Admin	Login → Admin Dashboard → Manage Bookings / Tournaments / Users / Reports
Customer	Login → Logout → Home
Customer	Login → Forgot Password → Enter Email → Reset Link (Email) → Set New Password → Login

Table 14: Navigation Flow by Role

6.5 Accessibility Considerations

The following WCAG 2.1 Level AA conformance requirements shall be implemented:

- All form inputs shall include associated <label> elements and ARIA attributes for screen reader compatibility.

- Colour contrast ratios shall meet the minimum of 4.5:1 for normal text and 3:1 for large text.
- All interactive elements (buttons, links, form fields) shall be keyboard-navigable and shall display a visible focus indicator.
- Error messages shall be communicated both visually and via ARIA live regions so that screen readers announce validation failures.
- All images and icons used decoratively shall have an empty alt attribute; informational images shall include descriptive alt text.
- The system shall not rely solely on colour to convey information; status indicators shall include text labels (e.g., Available, Occupied).

Chapter 7 Security Design

7.1 Authentication Mechanism

Authentication shall be implemented using JSON Web Tokens (JWT). Upon successful login, the User Service shall generate a signed JWT containing the user's ID, role, and expiration timestamp. The token shall be transmitted to the client and stored in a secure HTTP-only cookie to prevent XSS-based token theft.

- The cookie shall be configured with the following attributes:
  - HttpOnly
  - Secure
  - SameSite=Strict
- Token expiry shall be set to 60 minutes for customer sessions and 30 minutes for admin sessions.
- A refresh token mechanism shall be implemented to allow seamless session renewal without requiring the user to log in again. The refresh token shall also be stored in a secure HTTP-only cookie.
- For each request, the JWT shall be automatically included via cookies and validated by the API Gateway before forwarding the request to the appropriate microservice.

7.2 Authorisation Levels

Role-Based Access Control (RBAC) shall be enforced across all system endpoints. Two roles are defined: CUSTOMER and ADMIN.

Resource / Endpoint	Guest	CUSTOMER	ADMIN
Register / Login	✓	✓	✓
Book Gaming Slot	✗	✓	✗
Admin Walk-in Booking	✗	✗	✓
Register for Tournament	✗	✓	✗
Manage Tournament	✗	✗	✓
View Loyalty Points	✗	✓	✗
View Admin Dashboard	✗	✗	✓
Generate Reports	✗	✗	✓
Manage User Accounts	✗	✗	✓

Table 15: Role-Based Access Control Matrix

7.3 Data Protection Approach

- All customer passwords shall be stored exclusively as bcrypt hashes with a minimum cost factor of 12. Plain-text password storage is strictly prohibited.
- Personally identifiable information (PII) including names, email addresses, and phone numbers shall be stored in compliance with applicable data protection regulations.
- Payment card data shall not be stored within the Gaming Castle system; all card processing shall be delegated entirely to the integrated payment gateway (e.g., Stripe or PayHere), which is PCI DSS compliant.
- Database access credentials shall be stored as environment variables and shall not be hardcoded in source code or committed to version control.

7.4 Input Validation and Sanitisation Strategy

- All user inputs shall be validated on the client side (Angular reactive forms) and on the server side (Spring Boot Bean Validation) independently. Client-side validation alone shall not be relied upon as a security measure.
- All string inputs shall be sanitised to prevent Cross-Site Scripting (XSS) attacks. Angular's built-in DOM sanitiser shall be used on the frontend.
- All database queries shall use parameterised queries or JPA repositories to prevent SQL injection attacks.
- File uploads, if introduced in future iterations, shall be restricted to whitelisted MIME types and maximum file size limits.
- The system shall undergo an OWASP Top 10 vulnerability assessment prior to production deployment, in conformance with NFR12.

7.5 Encryption Standards

Context	Standard	Notes
Data in Transit	TLS 1.2 / 1.3	Enforced for all HTTP communications (HTTPS mandatory)
Password Storage	bcrypt (cost factor 12)	No plain-text or MD5/SHA-1 password hashing permitted
JWT Signing	HMAC-SHA256 (HS256)	Secret key stored in environment variable; RS256 optional for distributed scenarios
Database at Rest	AES-256	Cloud provider managed encryption for the PostgreSQL volume
Session Tokens	HTTP-only Secure Cookies	stored in HTTP-only cookies

Table 16: Encryption Standards

7.6 Account Security

- Brute-force protection: accounts shall be temporarily locked after 5 consecutive failed login attempts, as described in the Login sequence diagram.

- Password reset shall require verification via a time-limited token sent to the registered email address or phone number.
- Admin accounts shall be created exclusively by existing administrators; self-registration for the Admin role shall not be permitted.

Chapter 8 Deployment Design

8.1 Hosting Plan

The Gaming Castle system shall be deployed on Amazon Web Services (AWS) using AWS Free Tier for initial academic deployment and scalable paid tiers for production. The following AWS services shall be utilised:

Component	AWS Service	Purpose
Frontend	AWS S3 + CloudFront	Static Angular build served via CDN for low latency
API Gateway	AWS EC2 / ECS Fargate	Container orchestration for Spring Cloud Gateway
Microservices	AWS ECS Fargate	Serverless containers for each microservice
Database	AWS RDS (PostgreSQL)	Managed relational DB with automated backups
Load Balancer	AWS ALB	Application Load Balancer for traffic distribution

Container Registry	AWS ECR	Stores Docker images for each microservice

Table 17: Hosting Plan

8.2 Hardware Requirements

The system is designed for cloud-based deployment and does not require dedicated physical hardware. The recommended minimum specifications per containerised service instance are:

- vCPU: 0.5 vCPU per microservice instance (scalable to 2 vCPU under load)
- Memory: 512 MB RAM per instance (scalable to 2 GB under load)
- Storage: 50 GB SSD for the RDS PostgreSQL instance
- Network: Stable internet connectivity at the Gaming Castle admin terminal (minimum 10 Mbps)

8.3 Environment Specification

Environment	Infrastructure	Purpose
Development	Local machine; Docker Compose	Individual feature development and unit testing
Staging	AWS ECS Fargate (reduced capacity)	Integration testing, UAT, and pre-release validation
Production	AWS ECS Fargate (full capacity) + RDS	Live system accessible by Gaming Castle clients

Table 18: Environment Specification

8.4 CI/CD Pipeline Overview

A Continuous Integration / Continuous Deployment (CI/CD) pipeline shall be implemented using GitHub Actions to automate the build, test, and deployment process. The pipeline stages are as follows:

Stage	Tool	Action
1	GitHub Actions	Trigger on push to main or pull request to main branch
2	Maven / npm	Build backend microservices (mvn package) and frontend (ng build)
3	JUnit / Jest	Execute unit tests and integration tests; fail pipeline on test failure
4	Docker	Build Docker image for each modified microservice; tag with commit SHA
5	AWS ECR	Push tagged Docker images to AWS Elastic Container Registry
6	AWS ECS	Deploy updated container images to ECS Fargate (staging auto-deploy; production manual approval)
7	GitHub Actions	Notify team of deployment status via commit status and optional Slack alert

Table 19: CI/CD Pipeline Stages

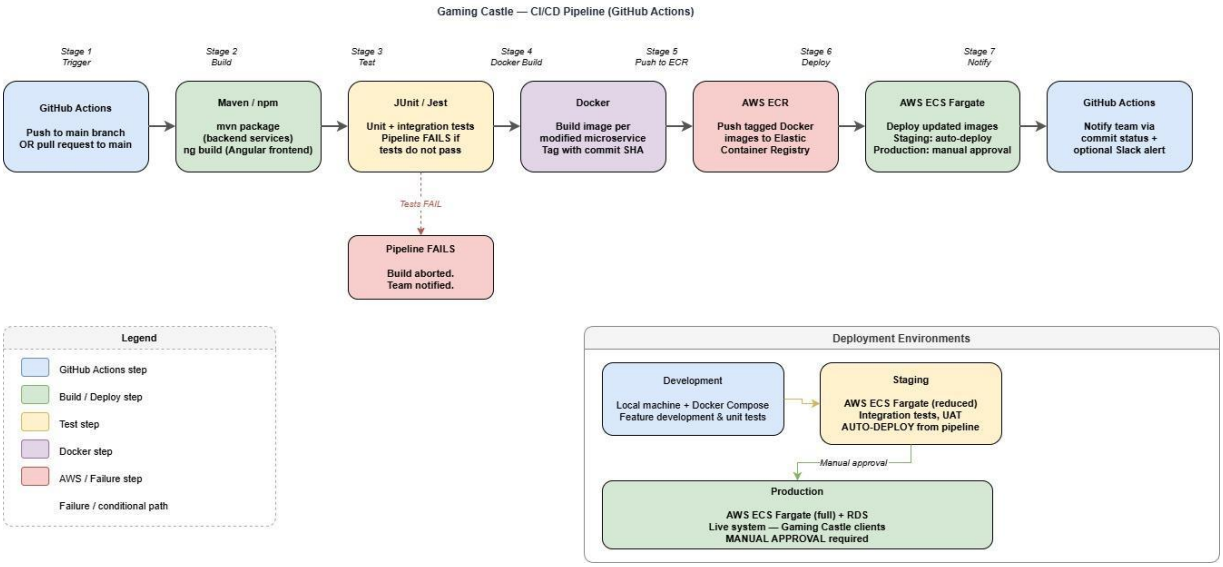

Figure 17: CI/CD Pipeline Diagram

8.5 Network Design

The network architecture shall follow AWS best practices for security and availability:

- The system shall be deployed within an AWS Virtual Private Cloud (VPC) divided into public and private subnets.
- The Angular frontend (S3/CloudFront) and the Application Load Balancer shall reside in the public subnet and be accessible from the internet.
- All microservice containers and the RDS PostgreSQL instance shall reside in private subnets and shall not be directly accessible from the internet.
- Traffic from the internet shall be routed through CloudFront (for the frontend) and the ALB (for API traffic), which forwards requests to the API Gateway container in the private subnet.
- Security Groups shall enforce least-privilege ingress and egress rules: only HTTPS (443) shall be exposed publicly; inter-service communication shall be restricted to the internal VPC CIDR range.

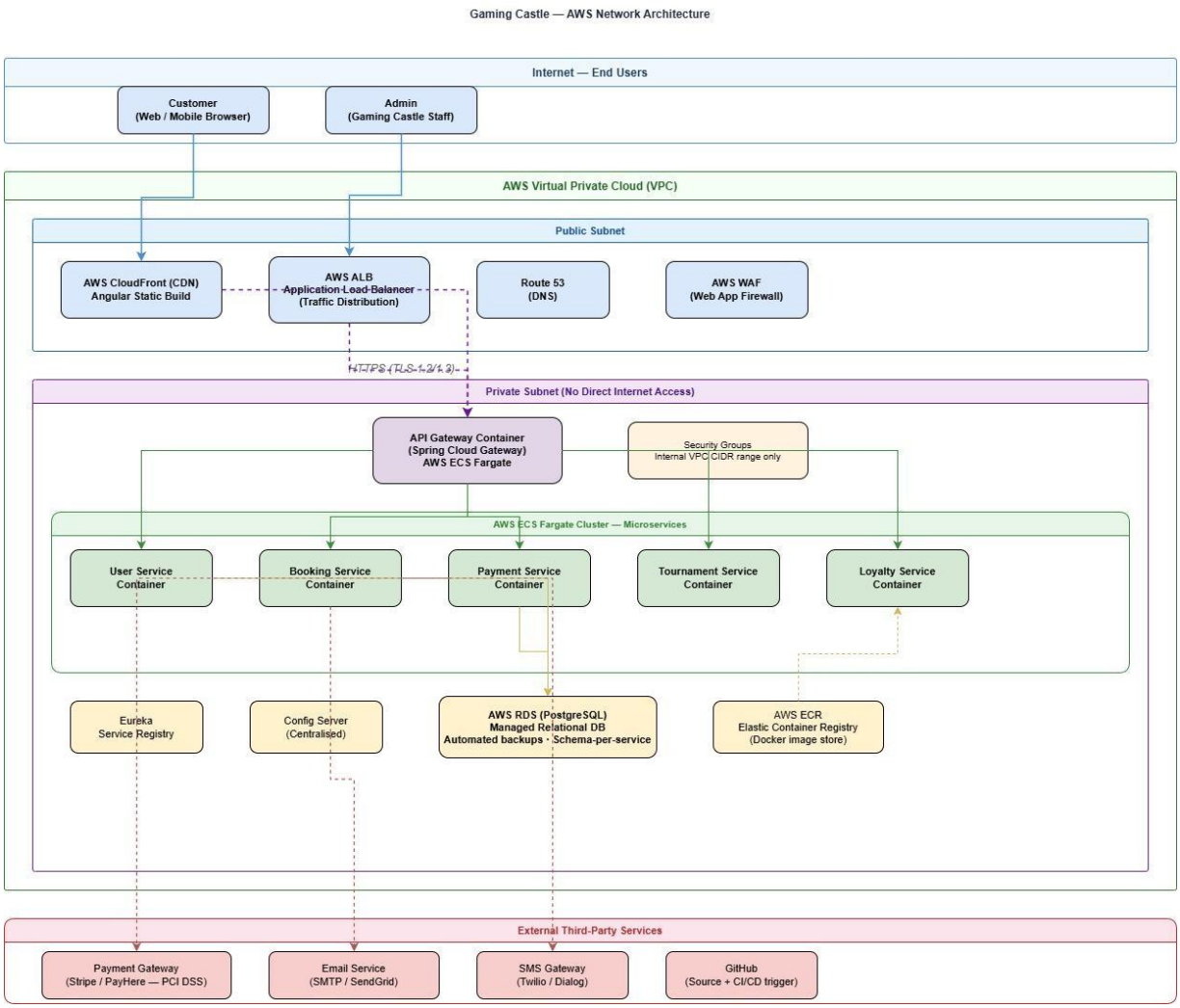

Figure 18: Network Architecture Diagram

Chapter 9 References

[1] IEEE, "IEEE Recommended Practice for Software Requirements Specifications," IEEE Std 830-1998, 1998.

[2] ISO/IEC, "Systems and software engineering – Systems and software Quality Requirements and Evaluation (SQuaRE)," ISO/IEC 25010:2011, 2011.

[3] OWASP Foundation, "OWASP Top Ten," 2021. [Online]. Available: https://owasp.org/www-project-top-ten/. (Accessed Apr. 10, 2026).

[4] W3C, "Web Content Accessibility Guidelines (WCAG) 2.1," 2018. [Online]. Available: https://www.w3.org/TR/WCAG21/. (Accessed Apr. 10, 2026).

[5] Spring, "Spring Boot Reference Documentation," Pivotal Software, 2024. [Online]. Available: https://docs.spring.io/spring-boot/docs/current/reference/html/. (Accessed Apr. 10, 2026).

[6] Docker Inc., "Docker Documentation," 2024. [Online]. Available: https://docs.docker.com/. (Accessed Apr. 10, 2026).

[7] Amazon Web Services, "AWS Architecture Center," 2024. [Online]. Available: https://aws.amazon.com/architecture/. (Accessed Apr. 10, 2026).

[8] M. Fowler, "Microservices," martinfowler.com, 2014. [Online]. Available: https://martinfowler.com/articles/microservices.html. (Accessed Apr. 10, 2026).